

HEXAGEN: A Model-Driven Engineering Platform for Automated Generation of Hexagonal Spring Boot Applications

Advanced Topics in Software Engineering
Universidad del Cauca

2026

Abstract

This document presents the detailed specification of HEXAGEN, a Model-Driven Engineering (MDE) platform for the automated generation of Spring Boot applications following hexagonal architecture. The project is designed as a 12-week hands-on course for 16 graduate/advanced undergraduate students organized in four teams. Students will design three metamodels and implement three model transformations using Eclipse Modeling Framework (EMF/Ecore), Sirius for graphical modeling, and the Epsilon family of languages (ETL, EGL, EVL). The platform will be validated through a Library System case study, demonstrating the feasibility of generating complete, executable applications from high-level conceptual specifications.

Contents

1	Introduction and Motivation	3
2	Related Work and Foundations	3
2.1	Model-Driven Engineering Foundations	3
2.2	Software Architecture and Generative Approaches	3
3	Project Architecture Overview	4
3.1	Three-Layer Metamodel Architecture	4
3.2	Transformation Chain	4
4	Metamodel Specifications	5
4.1	Conceptual Metamodel (CMM)	5
4.2	Hexagonal Metamodel (HMM)	5
4.3	Spring Boot Metamodel (SMM) - UNIFIED APPROACH	5
5	Team Organization and Responsibilities	5
5.1	Team Structure	5
6	Validation Case Study: Library System	6
6.1	Domain Specification	6
6.2	Validation Criteria	7

7	Implementation Roadmap	7
7.1	Week 1-2: Foundations and Training	7
7.2	Week 3-4: Metamodel Development	7
7.3	Week 5-7: Transformation Implementation	8
7.4	Week 8-9: Integration and Refinement	8
7.5	Week 10-11: Validation and Evaluation	8
7.6	Week 12: Final Delivery and Presentations	8
8	Technology Stack and Setup	8
8.1	Required Software	8
8.2	Recommended Project Structure	9
9	Evaluation Criteria	10
9.1	Project Evaluation (70%)	10
9.2	Individual Evaluation (30%)	10
10	Expected Outcomes	10
11	Knowledge Outcomes	11

1 Introduction and Motivation

Model-Driven Engineering (MDE) promotes the elevation of models from mere documentation to first-class artifacts in the software development lifecycle [Bezivin, 2005, Schmidt, 2006]. As stated by Bezivin [2005], MDE is fundamentally about “*integrating in a smooth way existing and mature bodies of knowledge through a shared understanding*”. This shared understanding is operationalized through metamodels and model transformations.

Despite the maturity of frameworks like Spring Boot and architectural patterns like hexagonal architecture, development remains largely manual, repetitive, and error-prone. Each new project requires developers to reinvent the same boilerplate code: JPA entities, repositories, service interfaces, implementations, REST controllers, DTOs, and mappers. Furthermore, architectural drift often occurs when manual implementations deviate from intended design principles [Favre, 2005].

HEXAGEN addresses this gap by providing a generative MDE infrastructure that captures architectural knowledge in reusable metamodels and transformations. The platform enables developers to specify their domain using high-level conceptual models and automatically obtain fully functional Spring Boot applications adhering to hexagonal architecture.

2 Related Work and Foundations

2.1 Model-Driven Engineering Foundations

The conceptual foundations of this project build upon the seminal work of Bezivin [2005], who established the key relationships between models, metamodels, and metametamodels in the MDE technical space. The four-layer architecture (M0-M3) provides the theoretical underpinning for our metamodeling approach.

Favre [2005] further emphasized that MDE is not equivalent to MDA (Model-Driven Architecture); rather, MDE encompasses a broader vision where models are integrated throughout the engineering lifecycle. Our approach aligns with this vision by incorporating domain modeling, architectural design, and implementation concerns into a coherent transformation chain.

2.2 Software Architecture and Generative Approaches

Hexagonal architecture, also known as ports and adapters architecture [Cockburn, 2005], promotes separation between the core domain and external concerns. This architectural style has gained significant adoption in enterprise Java development, particularly with Spring Boot [Richardson, 2018].

Previous work on generative approaches for enterprise applications includes the work of Kolovos et al. [2015] on the Epsilon platform, which provides a mature ecosystem for model management. Our project leverages Epsilon’s Transformation Language (ETL) for model-to-model transformations and Generation Language (EGL) for code generation.

3 Project Architecture Overview

3.1 Three-Layer Metamodel Architecture

The HEXAGEN platform is organized around three distinct metamodel layers, each capturing a different level of abstraction in the software development process:

1. **Conceptual Metamodel (CMM)**: Captures domain concepts, entities, attributes, operations, and business rules independent of any architectural or implementation concerns.
2. **Hexagonal Metamodel (HMM)**: Represents the system in terms of hexagonal architecture patterns, including domain entities, ports (input/output interfaces), and adapters (primary/secondary).
3. **Spring Boot Metamodel (SMM)**: Models implementation-level constructs specific to the Spring Boot framework, including annotated Java classes, components, and configurations.

3.2 Transformation Chain

The platform implements three core transformations:

1. **CMM2HMM**: Conceptual to Hexagonal transformation (ETL)
2. **HMM2SMM**: Hexagonal to Spring Boot transformation (ETL)
3. **SMM2Code**: Spring Boot to Java source code generation (EGL)

Figure 1 illustrates the complete transformation pipeline.

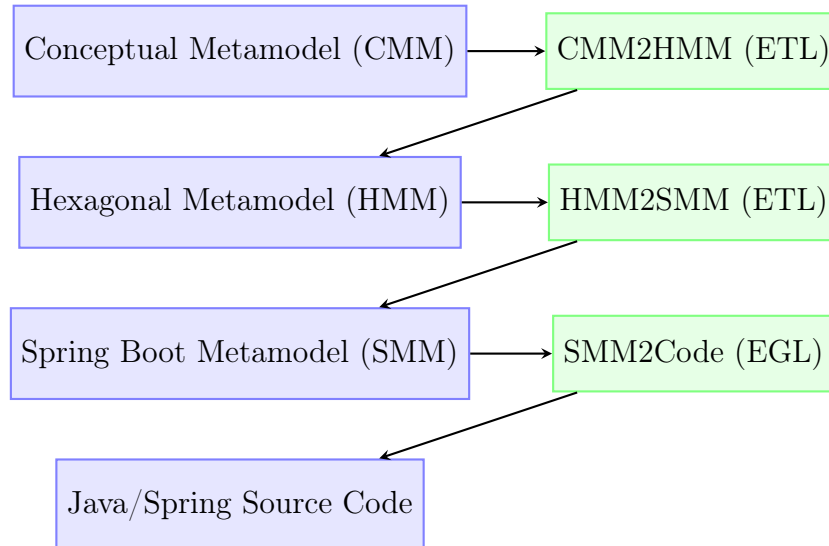


Figure 1: HEXAGEN Three-Layer Transformation Architecture

4 Metamodel Specifications

4.1 Conceptual Metamodel (CMM)

The Conceptual Metamodel provides a technology-agnostic representation of domain knowledge. It enables domain experts and analysts to specify system requirements in terms of business concepts.

4.2 Hexagonal Metamodel (HMM)

The Hexagonal Metamodel captures architectural decisions aligned with ports and adapters pattern. It separates the system into domain, application, and infrastructure concerns.

4.3 Spring Boot Metamodel (SMM) - UNIFIED APPROACH

After careful analysis of project constraints and learning objectives, we recommend a **single, unified Spring Boot Metamodel** rather than two separate metamodels. This decision is based on:

1. **Coherence:** Adapters (REST, events, persistence) are variants within the same metamodel, not separate metamodels
2. **Complexity management:** Two separate implementation metamodels would introduce unnecessary complexity in a 12-week timeframe
3. **Reusability:** A unified metamodel enables students to understand how different adapter types share common structures

The unified SMM captures:

- **Domain Implementation:** JPA entities with annotations, value objects
- **Application Layer:** Service interfaces and implementations, DTOs
- **Infrastructure Layer:** REST controllers, repository interfaces, event listeners, configuration classes
- **Common Concerns:** Exception handling, validation, mapping

5 Team Organization and Responsibilities

5.1 Team Structure

The 16 students are organized into four teams of four members each. This structure ensures clear responsibilities while maintaining interdependence and collaboration.

Team	Primary Responsibilities	Technologies	Dependencies
Team 1 CMM	• CMM Metamodel Design • CMM Graphical Editor • CMM2HMM Transformation	ECore, Sirius, EVL ETL	Provides CMM to Team 4 Receives validation feedback
Team 2 HMM	• HMM Metamodel Design • HMM Graphical Editor • HMM2SMM Transformation	ECore, Sirius, EVL ETL	Receives CMM from Team 1 Provides HMM to Team 4
Team 3 SMM	• SMM Metamodel Design • SMM Graphical Editor • SMM2Code Templates	ECore, Sirius, EVL EGL	Receives HMM2SMM from Team 4 Provides templates to Team 4
Team 4 Integration	• Transformation Integration • Pipeline Orchestration • Validation and Testing	ETL, EGL, EVL Java, Spring Boot	Receives all metamodels Executes transformations Delivers final system

Table 1: Team Organization and Responsibilities

6 Validation Case Study: Library System

To validate the HEXAGEN platform, all teams will collaborate on a common case study: a Library Management System. This domain provides sufficient complexity to demonstrate all aspects of the platform while remaining manageable within the project timeline.

6.1 Domain Specification

The Library System includes the following domain concepts:

Domain: Library

Entities:

- Book (isbn, title, author, publicationYear, status)
- User (id, name, email, membershipDate)
- Loan (id, book, user, loanDate, dueDate, returnDate)
- Reservation (id, book, user, reservationDate, status)

Services:

- LoanService (borrowBook, returnBook, renewLoan, checkOverdue)
- CatalogService (searchBooks, addBook, updateBook, removeBook)
- UserService (registerUser, updateUser, suspendUser)

Business Rules:

- Users cannot borrow more than 5 books simultaneously
- Loans are due 14 days after borrowing
- Overdue books incur a fine of 2500 COL per day
- Reserved books are held for 48 hours

6.2 Validation Criteria

Teams will evaluate the platform using the following metrics:

1. **Completeness:** Percentage of application code successfully generated
2. **Correctness:** Generated code compiles and passes unit tests
3. **Conciseness:** Ratio of generated code to manually written code
4. **Maintainability:** Effort required to modify metamodels and transformations
5. **Extensibility:** Effort to add new domain entities or adapter types

7 Implementation Roadmap

7.1 Week 1-2: Foundations and Training

- Eclipse Modeling Framework fundamentals
- ECore metamodeling
- Epsilon language family (ETL, EGL, EVL) tutorials
- Sirius graphical modeling basics
- Team formation and role assignment

7.2 Week 3-4: Metamodel Development

- Team 1: CMM specification and ECore implementation
- Team 2: HMM specification and ECore implementation
- Team 3: SMM specification and ECore implementation
- Team 4: Integration strategy definition
- Validation constraints in EVL
- Sirius diagram editors for each metamodel

7.3 Week 5-7: Transformation Implementation

- Team 1: CMM2HMM ETL transformation rules
- Team 2: HMM2SMM ETL transformation rules
- Team 3: SMM2Code EGL templates
- Team 4: Integration infrastructure and testing

7.4 Week 8-9: Integration and Refinement

- Full pipeline integration
- Library System case study implementation
- Debugging and transformation refinement
- Performance optimization

7.5 Week 10-11: Validation and Evaluation

- Systematic validation using Library System
- Metrics collection and analysis
- Documentation and user guide
- Presentation preparation

7.6 Week 12: Final Delivery and Presentations

- Project demonstration
- Code and artifact submission
- Technical presentations
- Retrospective and lessons learned

8 Technology Stack and Setup

8.1 Required Software

- **Eclipse Modeling Tools 2023-12**: Integrated development environment
- **EMF - Eclipse Modeling Framework 2.30+**: Core metamodeling
- **Epsilon 2.5+**: ETL, EGL, EVL languages
- **Sirius 7.0+**: Graphical model editors
- **Xtext 2.30+**: Optional for textual DSL (if time permits)

- **Git/GitHub:** Version control and collaboration
- **Maven 3.9+:** Build automation for generated code
- **JDK 17+:** Java development kit
- **Spring Boot 3.1+:** Target framework

8.2 Recommended Project Structure

```

hexagen/
  metamodels/
    conceptual/
      model/cmm.ecore
      edit/cmm.edit
      editor/cmm.editor
    hexagonal/
      model/hmm.ecore
      edit/hmm.edit
      editor/hmm.editor
    springboot/
      model/smm.ecore
      edit/smm.edit
      editor/smm.editor
  transformations/
    cmm2hmm/
      CMM2HMM.etl
    hmm2smm/
      HMM2SMM.etl
    smm2code/
      templates/
        SMM2Code.egl
  validation/
    cmm.evl
    hmm.evl
    smm.evl
  tests/
    library-system/
      models/
        generated/
  docs/
    ...

```

9 Evaluation Criteria

9.1 Project Evaluation (70%)

Criterion	Description	Weight
Metamodel Quality	Completeness, correctness, and expressiveness of ECore models	15%
Transformation Correctness	ETL/EGL transformations produce expected outputs	20%
Code Generation	Generated Spring Boot applications compile and execute	25%
Validation	EVL constraints correctly identify model violations	10%
Documentation	Technical documentation and user guides	10%
Collaboration	Effective use of Git, team coordination, integration	10%
Innovation	Extensions beyond baseline requirements	10%

Table 2: Project Evaluation Criteria

9.2 Individual Evaluation (30%)

- Individual contributions to team deliverables
- Peer evaluations and team feedback
- Weekly progress reports
- Technical presentation quality

10 Expected Outcomes

Upon successful completion, the HEXAGEN project will deliver:

1. **Three validated metamodels:** CMM, HMM, SMM with ECore implementations and Sirius graphical editors
2. **Three model transformations:** CMM2HMM (ETL), HMM2SMM (ETL), SMM2Code (EGL)
3. **Constraint validation:** EVL rules ensuring model well-formedness at each level
4. **Integration pipeline:** End-to-end automated generation system
5. **Case study demonstration:** Fully generated Library System application
6. **Reusable infrastructure:** Platform applicable to multiple domains
7. **Technical documentation:** Comprehensive documentation of metamodels, transformations, and usage

11 Knowledge Outcomes

HEXAGEN represents a substantial yet achievable MDE project for a 16-student team over 12 weeks. By adopting a three-layer metamodel architecture and leveraging mature Eclipse technologies, students will gain practical experience in:

- Designing domain-specific metamodels
- Implementing model transformations with Epsilon
- Generating production-quality code
- Working in coordinated teams on complex MDE projects
- Applying MDE principles to real-world software engineering problems

The project aligns with both academic objectives (understanding MDE foundations) and industrial relevance (automated generation of enterprise applications). The Library System case study provides concrete validation while demonstrating the platform’s applicability beyond the specific domain.

References

- Bézivin, J. (2005). On the unification power of models. *Software & Systems Modeling*, 4(2), 171–188.
- Bézivin, J., Jouault, F., Rosenthal, P., & Valduriez, P. (2005). Modeling in the large and modeling in the small. In *European Conference on Model Driven Architecture*, 33–46.
- Cockburn, A. (2005). *Hexagonal architecture*. Available at: <https://alistair.cockburn.us/hexagonal-architecture/>
- Favre, J. M. (2005). Foundations of meta-pyramids: Languages vs. metamodels. In *Proceedings of Dagstuhl Seminar on Language Engineering for Model-Driven Software Development*.
- Kolovos, D. S., Rose, L. M., Paige, R. F., & García-Domínguez, A. (2015). The Epsilon book. *Structure and Semantics of Model Management*.
- Richardson, C. (2018). *Microservices patterns: with examples in Java*. Manning Publications.
- Schmidt, D. C. (2006). Model-driven engineering. *IEEE Computer*, 39(2), 25–31.